

A.

1.1. a), c), d)

1.2. SET implemented on a Dynamic Array

↳ no pos., no order, unique

⇒ ~~remove~~ elem. from a position (bc ~~positions~~)

b)

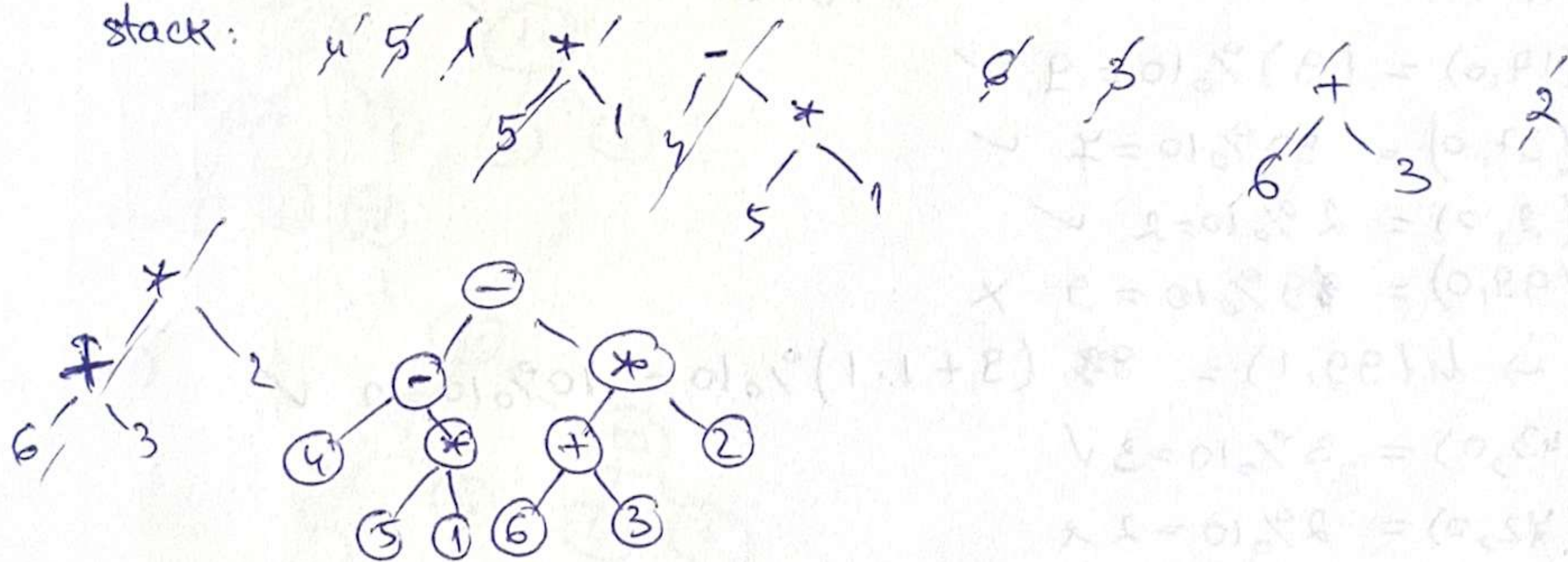
1.3. HT, 50 pos., collision res. with sep. chaining

→ (iv) nr. of elems. can be inserted.

⇒ a) b) c) d)

1.4. 4 5 1 * - 6 3 + 2 * -

stack:



B. 2.1.) HT, $m=10$, open addressing, double hashing, div. method,
 $h_2(k) = 1 + (k \% 9)$

insert: 19, 37, 2, 99, 43, 72, 54, 17

Open addressing → probe sequence

- double hashing: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \% m$

Lecture ex: $m=17$, $h_1(k) = k \% m$; $h_2(k) = (1 + (k \% 16))$

insert: 75, 12, 109, 43, 22, 18, 55, 81, 92, 27, 13, 16, 39.

Key	75	12	109	43	22	18	55	81	92	27	13	16	29
$h^1(\text{key})$	7	12	7	9	5	1	4	13	7	10	13	16	5
$h^2(\text{key})$	12	13	14	12	7	3	8	2	13	12	14	1	8

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
				109			75					12				

$h^3 = (7 + 10 + 12) \% 17 = 7 \checkmark$

$h(75, 0) = (7 + 0 \cdot 12) \% 17 = 7 \checkmark$

$h(12, 0) = (12 + 0 \cdot 13) \% 17 = 12$

$h(109, 0) = (7 + 0 \cdot 14) \% 17 = 7$; $h(109, 1) = (7 + 1 \cdot 14) \% 17 = 21 \% 17 = 4$...

7) merge two sorted SL

Key	19	37	2	99	43	72	54	17
$h_1(\text{Key})$	9	7	2	9	3	2	4	7
$h_2(\text{Key})$	2	2	3	1	8	1	1	9

$$19 \% 10 = 9$$

$$1 + (19 \% 9) = 1 + 1 = 2$$

$$1 + (37 \% 9) = 1 + 1 = 2$$

$$1 + (2 \% 9) = 1 + 2 = 3$$

$$1 + (99 \% 9) = 1 + 0 = 1$$

$$1 + (43 \% 9) = 1 + 7 = 8$$

$$1 + (72 \% 9) = 1$$

$$1 + (54 \% 9) = 1$$

$$1 + (17 \% 9) = 1 + 8 = 9$$

0	1	2	3	4	5	6	7	8	9
99		2	43	72	54	17	37		19

$$h(k, i) = (h^1(k) + i \cdot h^2(k)) \% m$$

$$h(19, 0) = (9) \% 10 = 9 \checkmark$$

$$h(37, 0) = 37 \% 10 = 7 \checkmark$$

$$h(2, 0) = 2 \% 10 = 2 \checkmark$$

$$h(99, 0) = 99 \% 10 = 9 \times$$

$$\hookrightarrow h(99, 1) = (9 + 1 \cdot 1) \% 10 = 10 \% 10 = 0 \checkmark$$

$$h(43, 0) = 3 \% 10 = 3 \checkmark$$

$$h(72, 0) = 2 \% 10 = 2 \times$$

$$\hookrightarrow h(72, 1) = (2 + 1 \cdot 1) \% 10 = 3 \times$$

$$\hookrightarrow h(72, 2) = (2 + 2 \cdot 1) \% 10 = 4 \checkmark$$

$$h(54, 0) = 4 \times$$

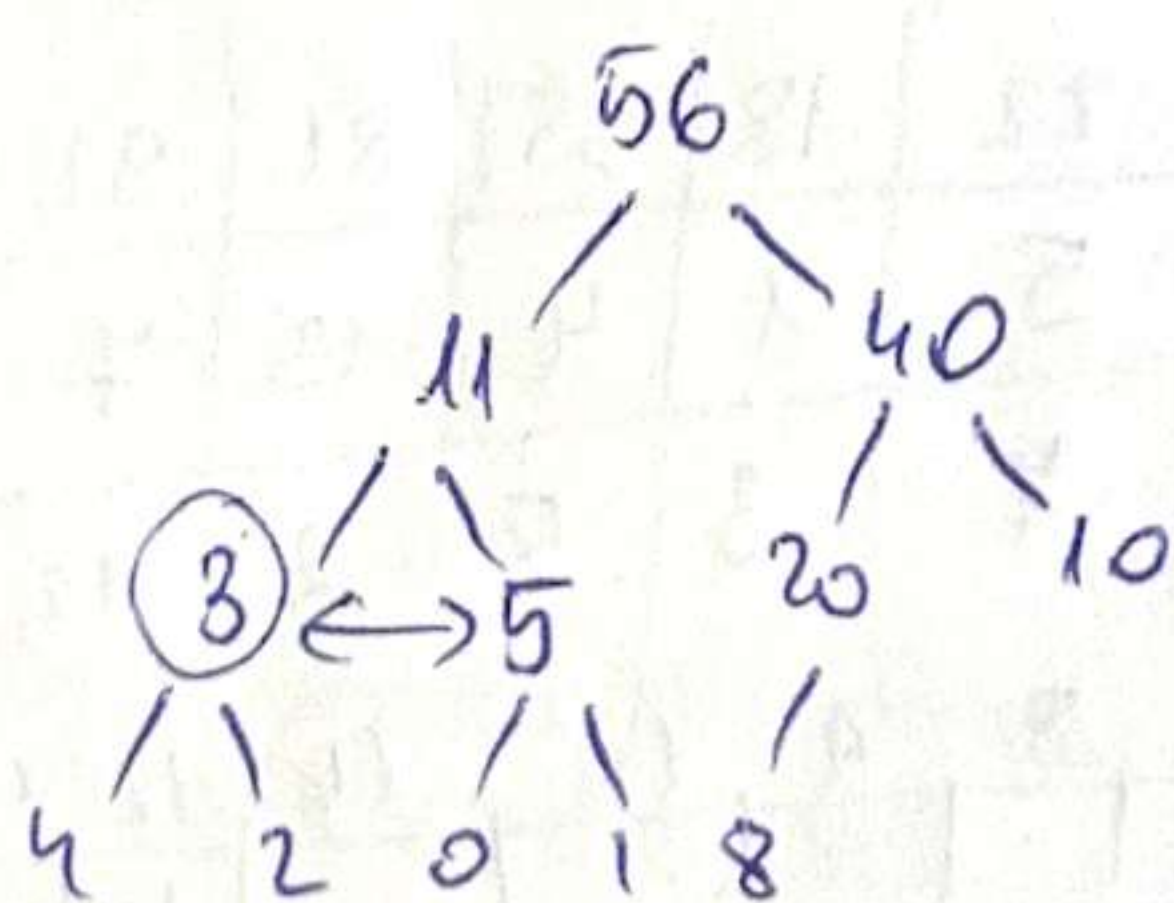
$$\hookrightarrow h(54, 1) = (4 + 1 \cdot 1) \% 10 = 5 \checkmark$$

$$h(17, 0) = 7 \% 10 = 7 \times$$

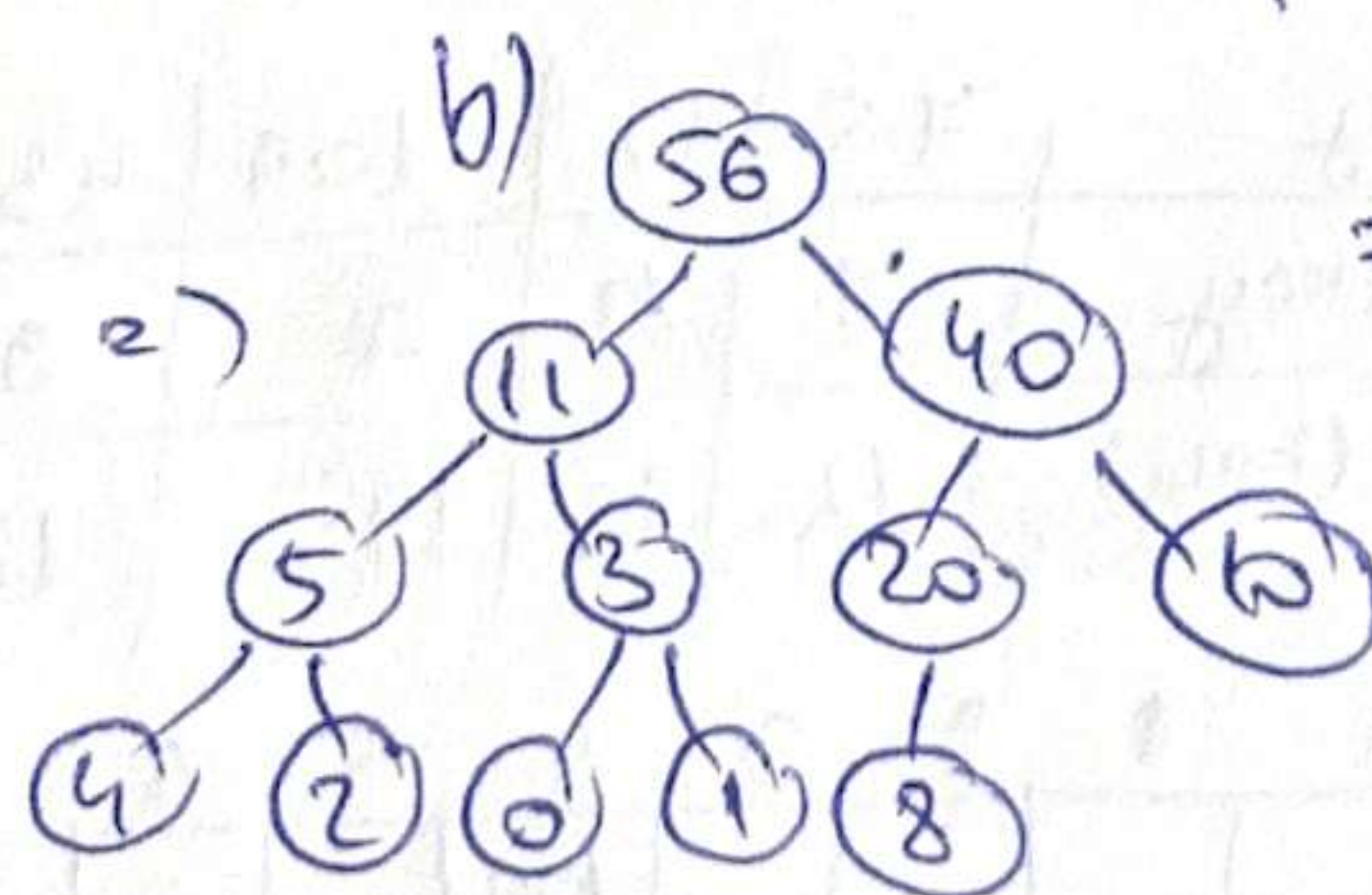
$$\hookrightarrow h(17, 1) = (7 + 1 \cdot 9) \% 10 = 16 \% 10 = 6 \checkmark$$

2.2. a) Is $[56, 11, 40, 3, 5, 20, 10, 4, 2, 0, 1, 8]$ a heap? No.

swap consecutive



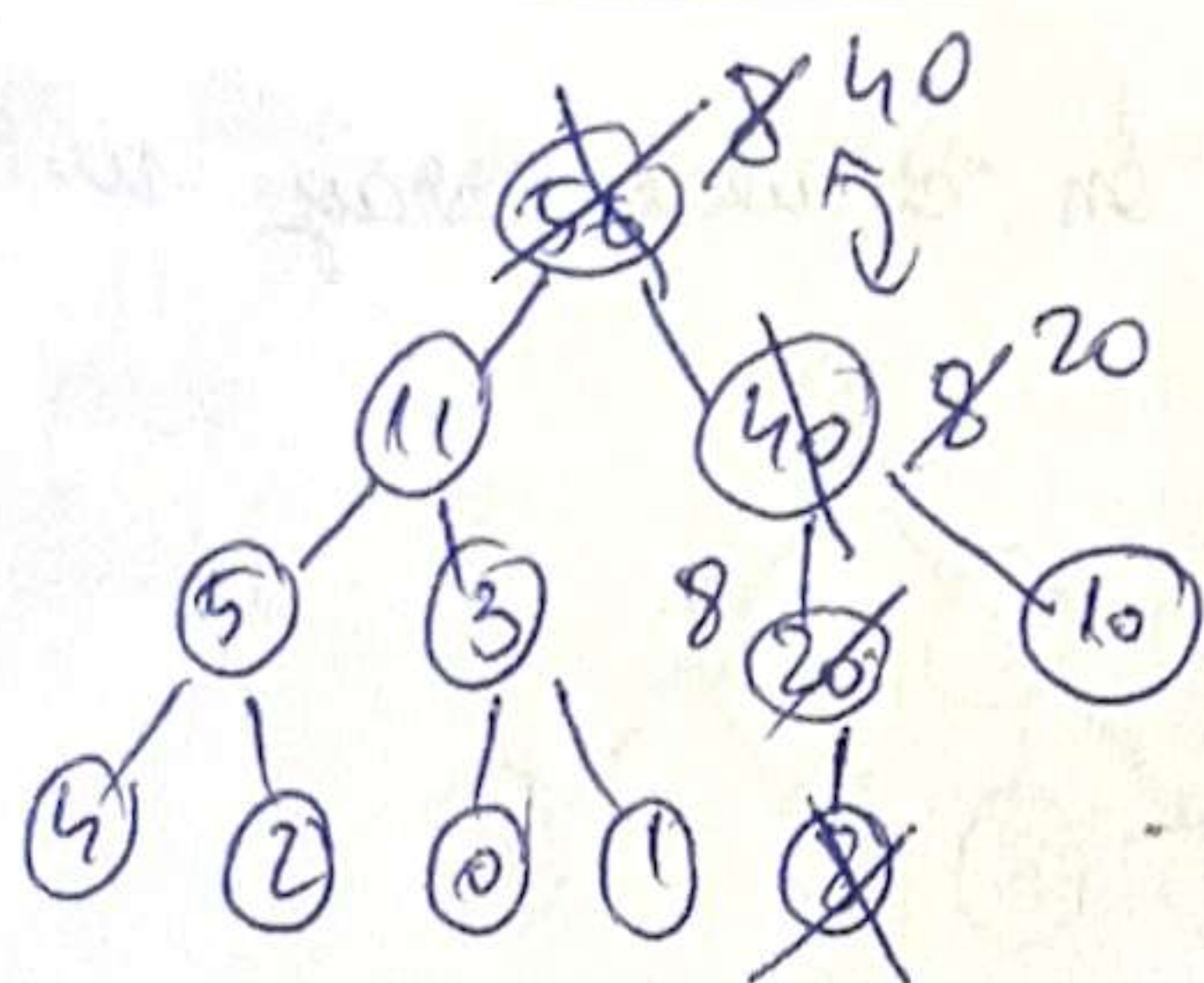
5 $\leftrightarrow$ 3



$\Rightarrow$ valid max-heap.

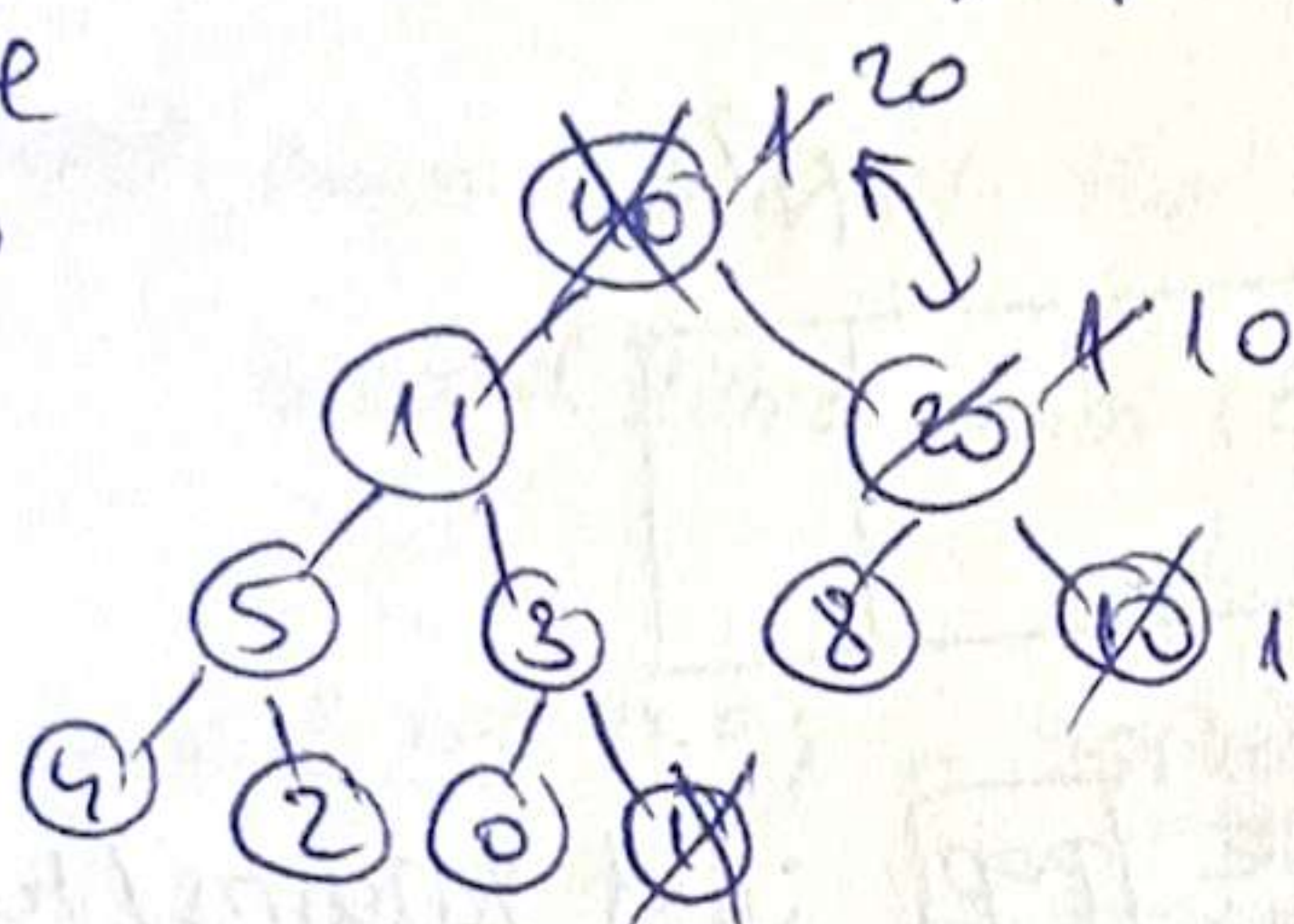
5 $\leftrightarrow$ 3

1) remove



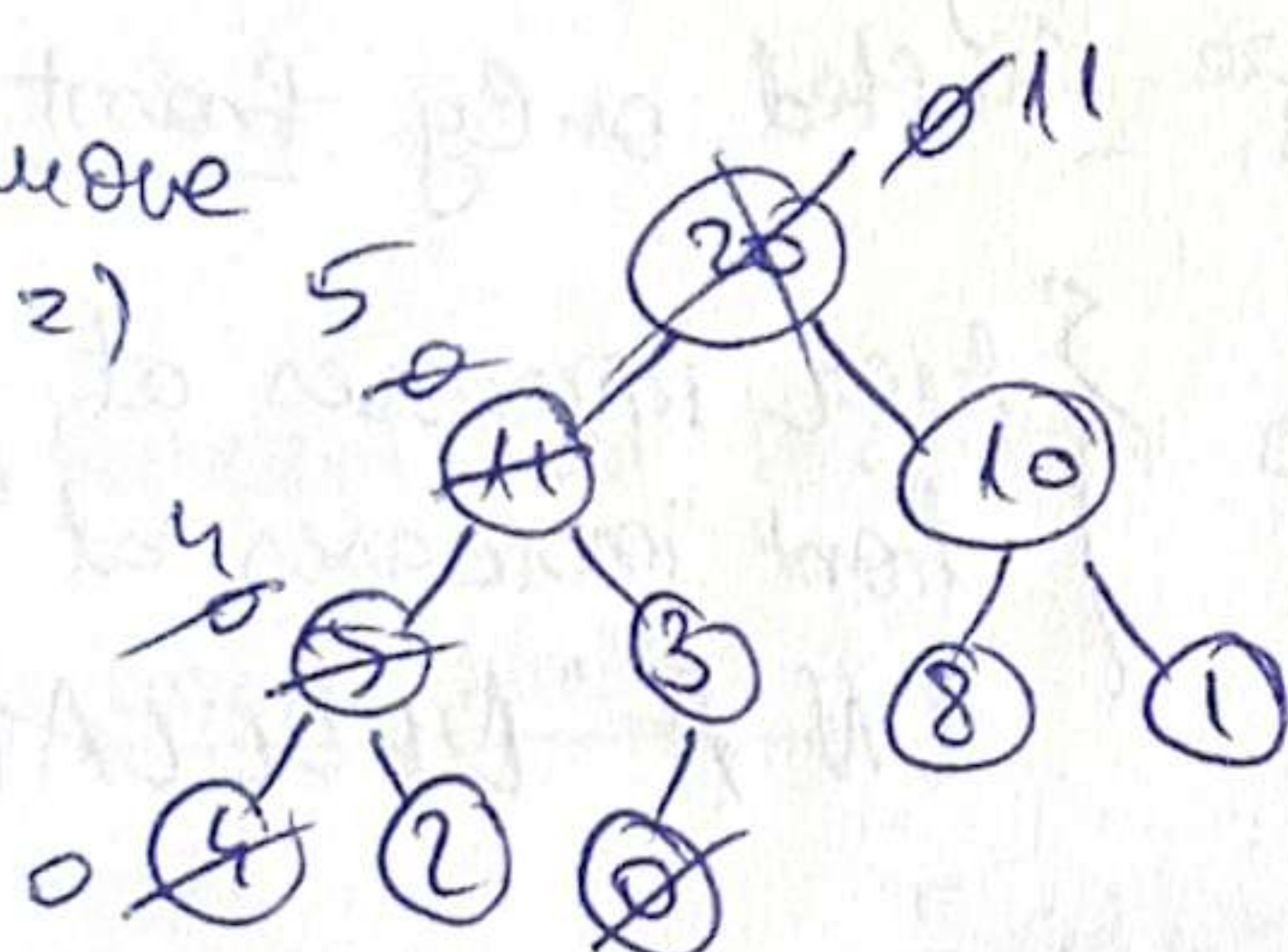
bubble down

remove
2)

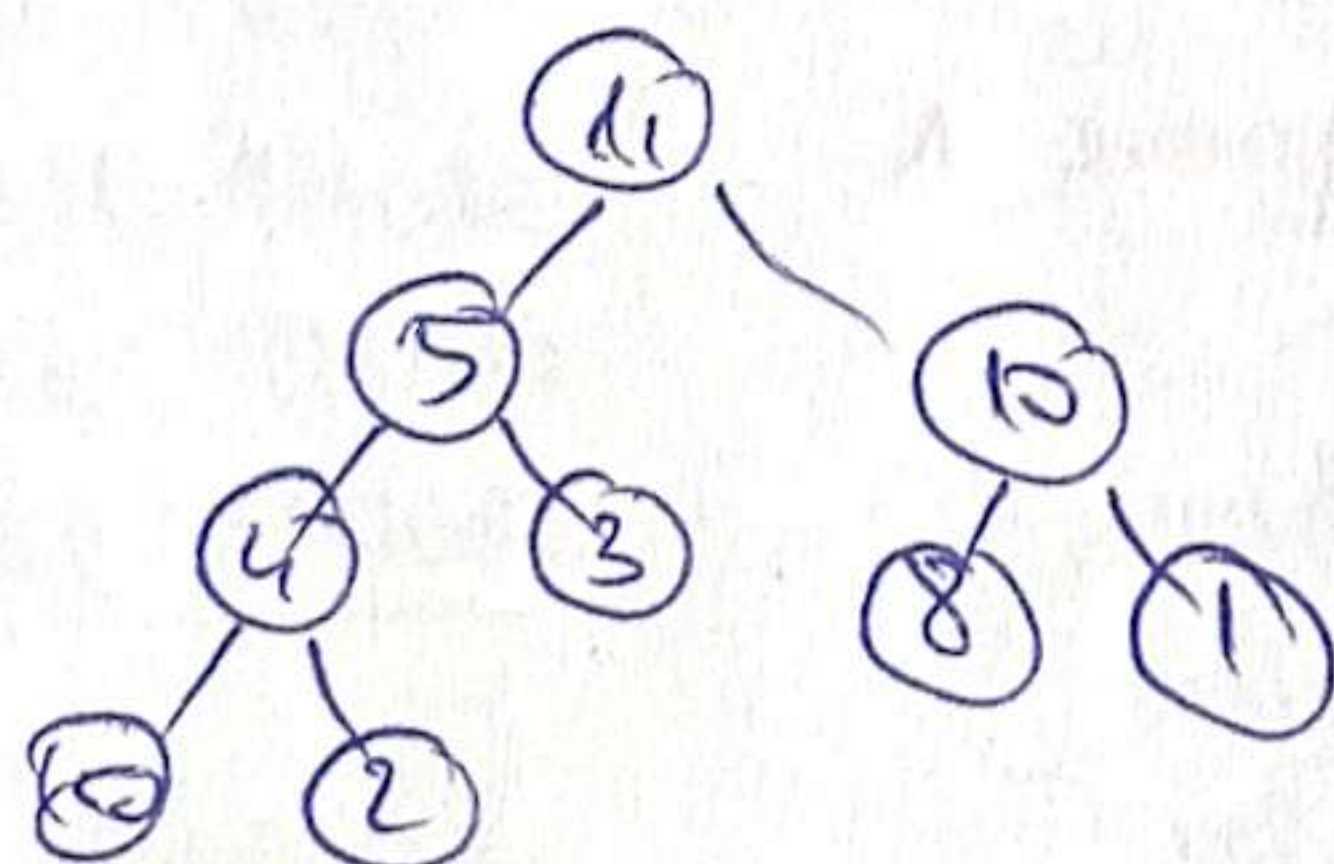


=)

remove
2)



2)

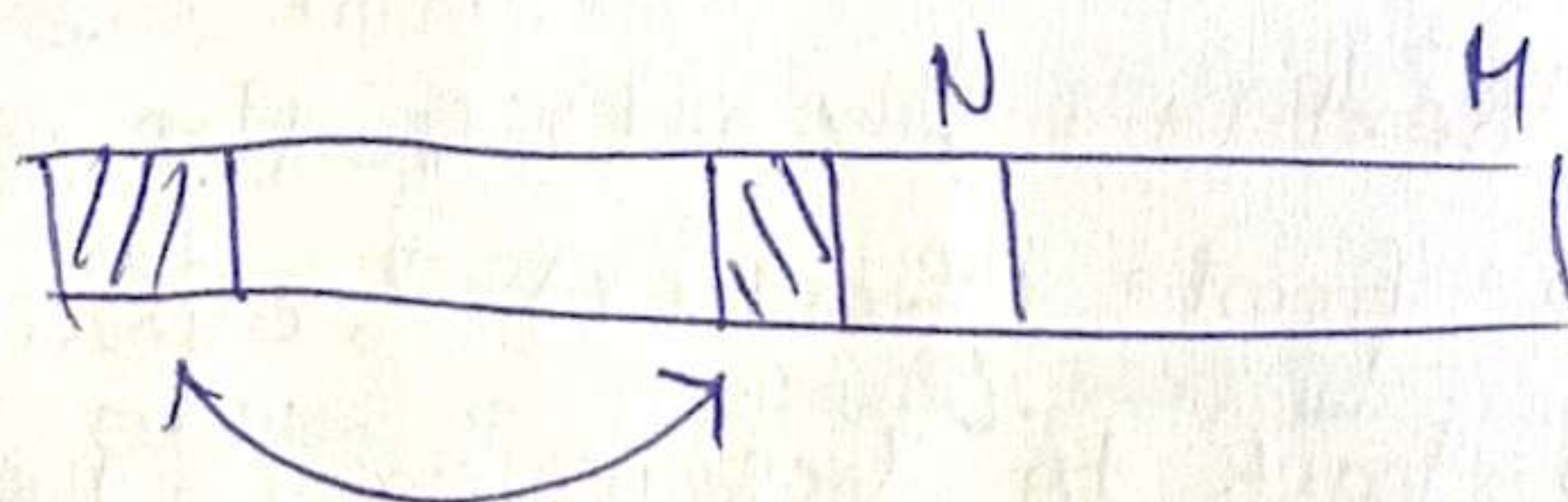


2.3). tightest bound for the WC.

algorithms $\rightarrow$ most efficient implem.

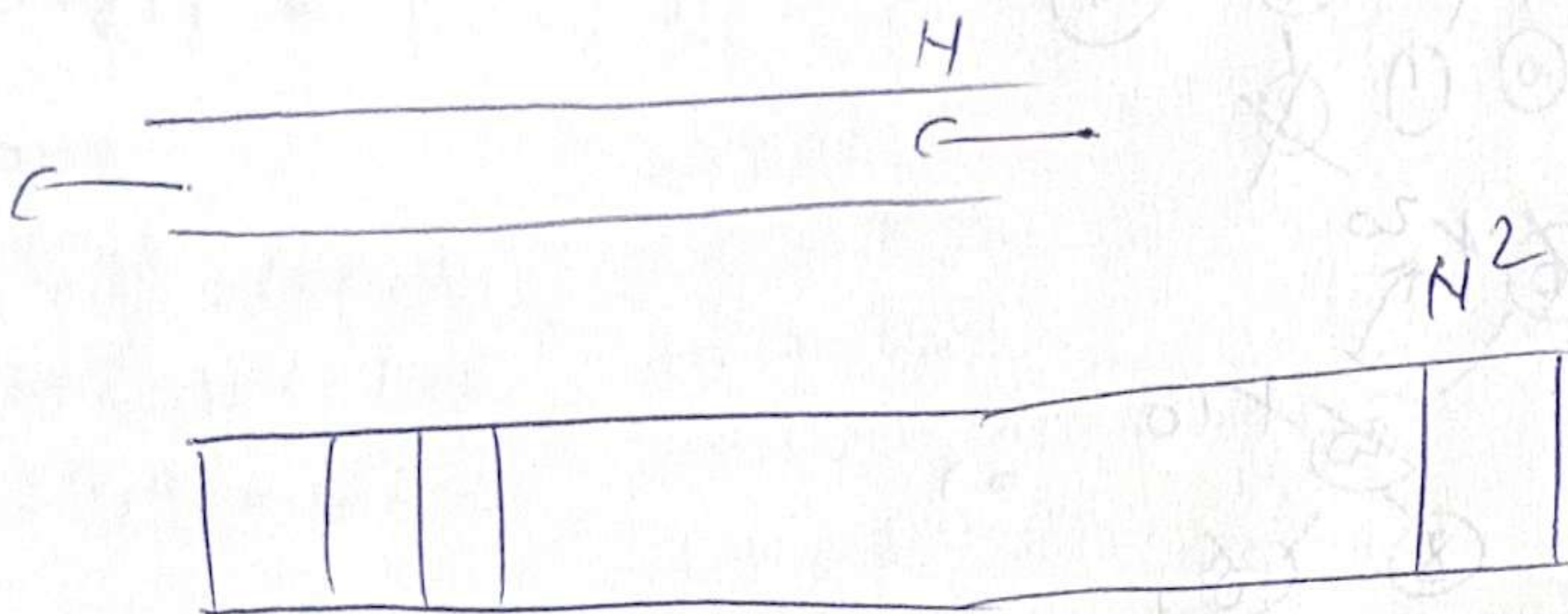
array-based-structures $\rightarrow$ large enough array.

a) swap first elem. with middle elem. in an array of size N containing N elems.



$\Theta(1)$ because array access by index is $O(1)$.
 $a[i]$, $a[mid]$.

b) pop elem. from queue represented on circular array with N^2 pos.
Queue has N elems.



$\Theta(1)$ because circular-array queue pop just returns/removes the front element (no elements are shifted, only front)

pop() on a circular array:

Index:	0	1	2	3	4
	A	B	C	-	-
	↓		↓		
	front		rear		

pop() → remove A = ignore A
→ front++ = 1.

Index:	0	1	2	3	4
	A	B	C	-	-
		↑	↑		
		front	rear		

rear increases at push
front increases at pop
ON A CIRCULAR ARRAY

"front modulo capacity" = eventually the front reaches the last position.

Index:	0	1	2	3	4
	A	B	C	-	-
					↑
					front

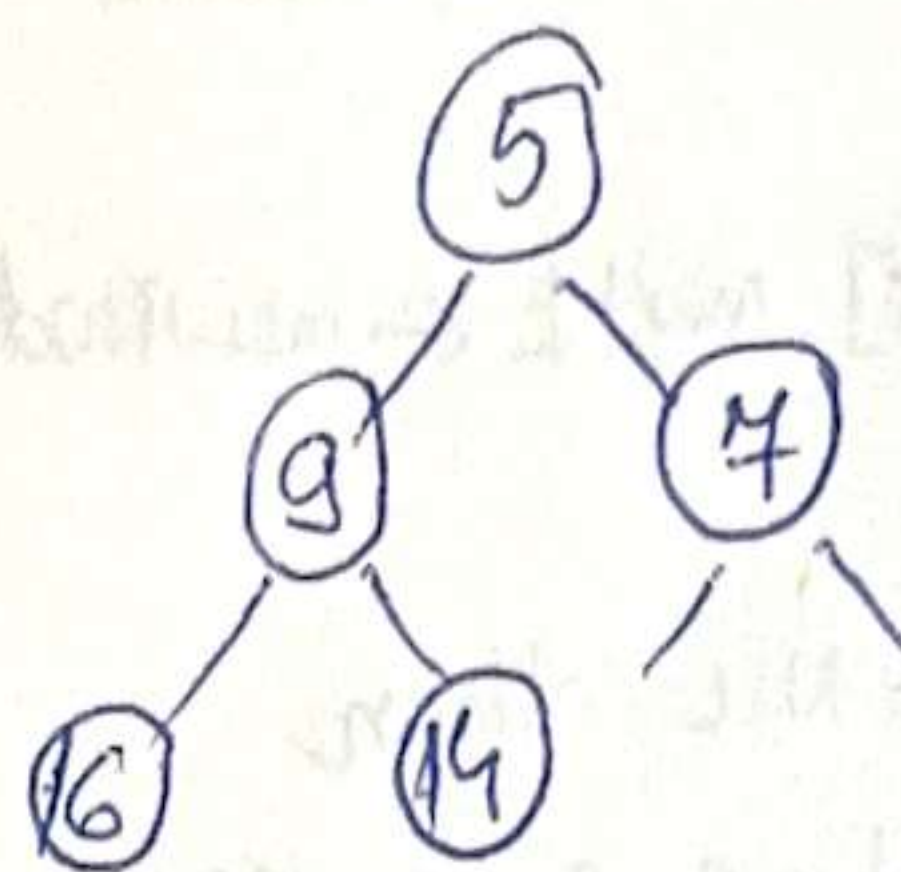
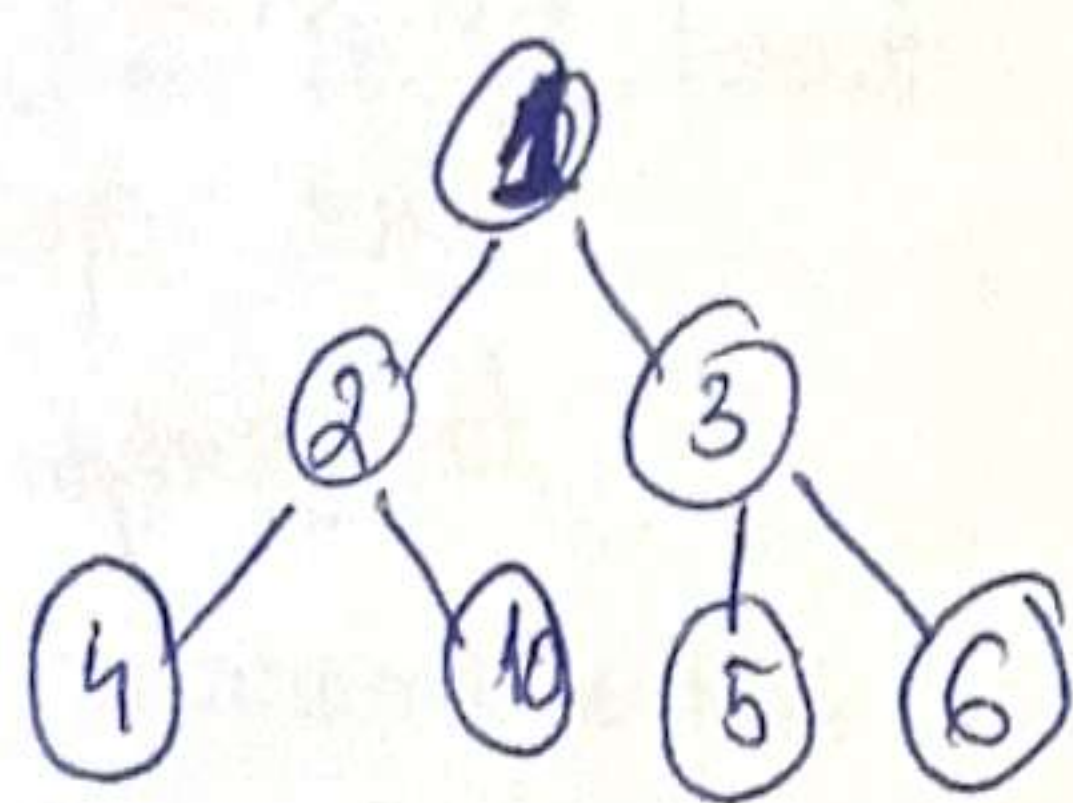
pop() → front++ = 5 (outside the array)

⇒ front = (front + 1) % capacity = 5 % 5 = 0

⇒ back to beginning (0)

1) second min. elem. in a min heap of N elems.

(1)
ROOT.
(first)



$\Rightarrow$ second min = one of the root's children. $\Rightarrow \Theta(1)$

Second min = $\min(\text{root.left}, \text{root.right})$.

2. 3) Linked HT (DS) - sorted order ($\leq$)

- search, add, remove.

- iterator gets elems. in sorted order: $\Theta(1)$

a) Representation iterator + container.

Lecture 11 Linked HT:

Node

info: TKey

nextH: $\uparrow$ Node

nextL: $\uparrow$ Node

prevL: $\uparrow$ Node

for efficient removal
(DLL)

Sorted Linked HT:

m: Integer

T: ($\uparrow$ Node)[]

h: TFunction

head: $\uparrow$ Node

tail: $\uparrow$ Node

rel: Relation

Iterator:

ht: SortedLinkedHT

current: $\uparrow$ Node.

← added! If we don't add it,
we're left with the
insertion order.

b) add $\rightarrow$ BC, WC, T;

subalgorithm add(ht, e) is

newNode $\leftarrow$ allocate

[newNode].info $\leftarrow$ e

[pos] $\leftarrow$ ht.h(e)

[newNode].nextH $\leftarrow$ ht.T[pos]

prev $\leftarrow$ Nil

current $\leftarrow$ ht.head

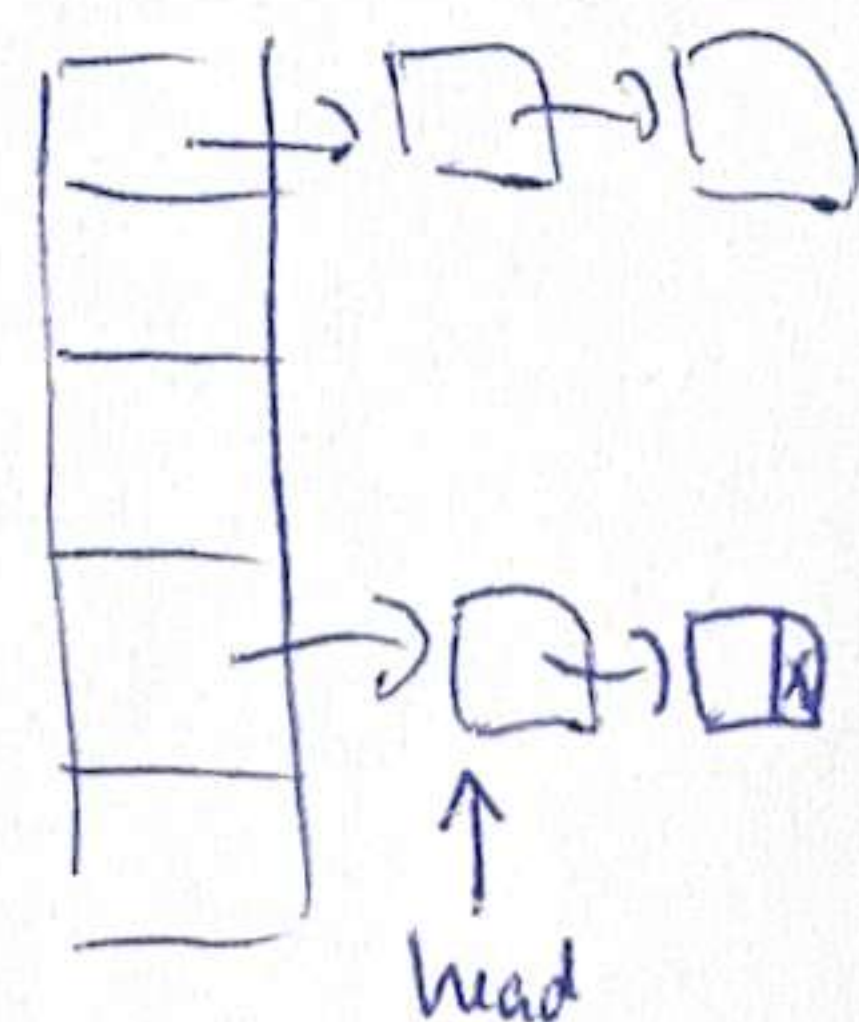
while current $\neq$ Nil and ht.rel([current].info, e) ex

┌ prev $\leftarrow$ current

└ current $\leftarrow$ [current].next

[newNode].prevL $\leftarrow$ prev

[newNode].nextL $\leftarrow$ current



insert at the
beginning of
the LL of that
slot.

In order for the iterator to have operations $\Theta(1)$, we need to keep the lists sorted.

```
if prev = Nil then
    ht.head  $\leftarrow$  newNode
else
    [prev].nextL  $\leftarrow$  newNode
    [
if current = Nil then
    ht.tail  $\leftarrow$  newNode
else
    [current].prevL  $\leftarrow$  newNode
    ]
    ]
```

Bc: $\Theta(1)$ $\rightarrow$ new node inserted at the beginning of the sorted list.
wc: $\Theta(n)$ $\rightarrow$ scans whole list for the correct place to insert.
T: $O(n)$

c) search, remove, Bc, wc, T.

- search(e) $\rightarrow$ pos $\leftarrow$ h(e)
 $\rightarrow$ search e only in T[pos] collision chain.
 $\rightarrow$ Bc: $\Theta(1)$ (first)
wc: $\Theta(n)$ traverse all / at the end / not found.
T: $O(n)$

- remove(e) $\rightarrow$ search: $O(n)$
 $\rightarrow$ remove from hash chain } ??
 $\rightarrow$ remove from linked list } ??
Bc: $\Theta(1)$
wc: $\Theta(n)$
T: $O(n)$

d) Iterator op: first
valid
getCurrent
next

subalgorithm first(it) is

- it.current $\leftarrow$ it.ht.head

function valid(it) is

- valid $\leftarrow$ it.current $\neq$ Nil

function getCurrent(it) is

- if not valid(it) then
 - @ throw --
- getCurrent $\leftarrow$ ~~current~~ [it.current].info

Subalgorithm next(it) is

- if not valid(it) then
 - @ throw --
- it.current $\leftarrow$ [it.current].nextL

nextL, not nextH because the iterator traverses the sorted LL, not the hash chains.

$\Theta(1)$

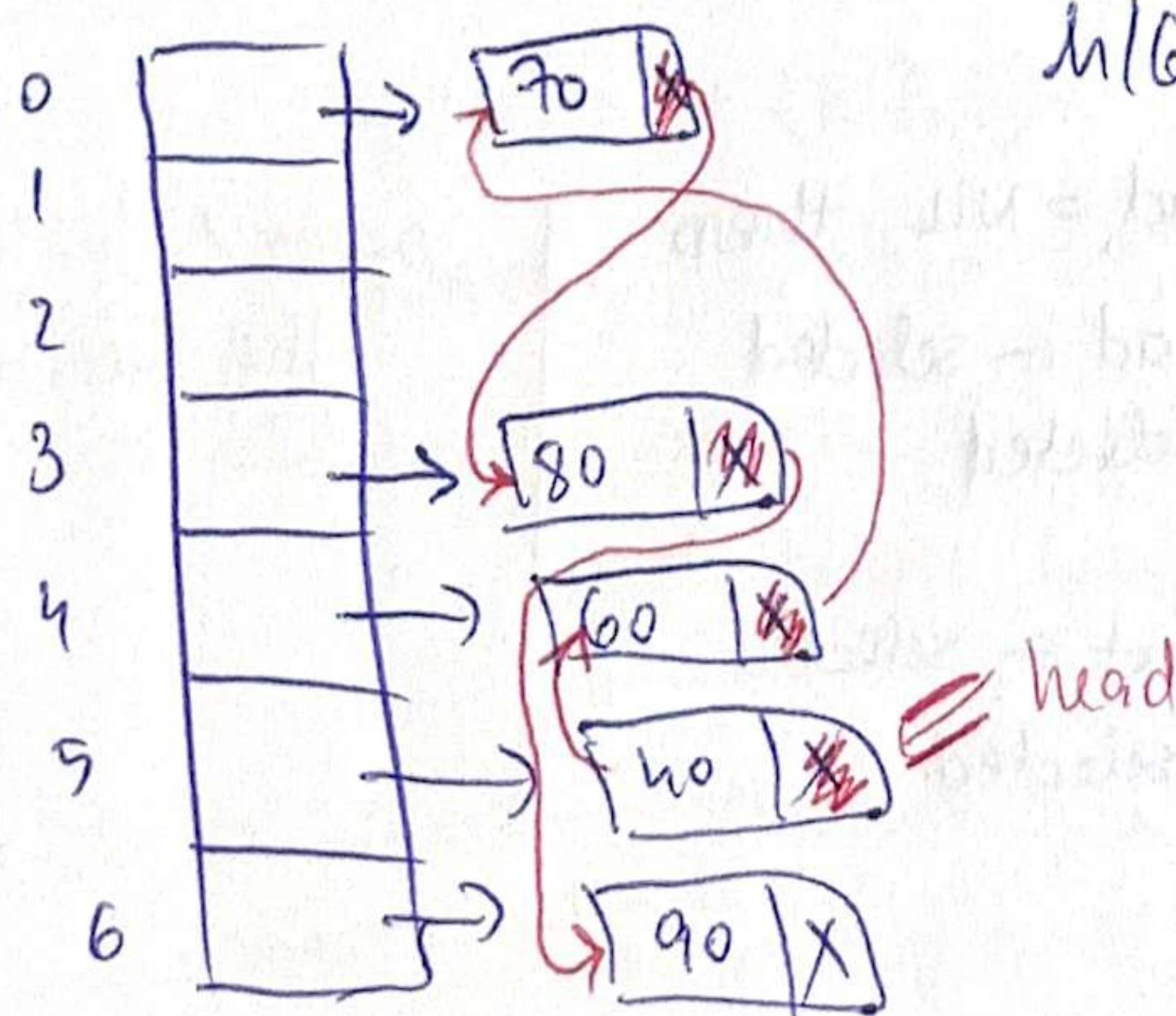
e) 90, 40, 70, 80, 60
m = 7
div. method.

$90 \% 7 = 6 = h(90)$
 $h(40) = 5$
 $h(70) = 0 \checkmark$
 $h(80) = 3 \checkmark$
 $h(60) = 4 \checkmark$

$$\begin{array}{r} 40 \overline{) 7} \\ 20 \\ \underline{12} \\ 6 \end{array}$$

$$\begin{array}{r} 80 \overline{) 7} \\ 20 \\ \underline{10} \\ 7 \\ 3 \end{array}$$

2.8



Sorted linked order: 40, 60, 70, 80, 90

NextH is only for traversing one collision chain after computing the bucket with $h(key)$

4) merge two sorted SLL.

a) SLL repres.

SLLNode

info: TElem

next: $\uparrow$ SLLNode

SLL

head: $\uparrow$ SLLNode

c) $m_1 = \text{size of sll1}$
 $m_2 = \text{size of sll2}$

result: $m_1 + m_2 = n$

BC: $\Theta(\min(m_1, m_2))$ ~~BC~~

↳ when all elems. of one list come before all elems. of the other list.

b) → no new nodes allocated
→ no lists kept

WC: $\Theta(m_1 + m_2)$

↳ when all elems are interleaved and we must compare all nodes.

subalgorithm merge (sll1, sll2) is

$p_1 \leftarrow \text{sll1.head}$

$p_2 \leftarrow \text{sll2.head}$

result.head $\leftarrow \text{NIL}$

last $\leftarrow \text{NIL}$

while $p_1 \neq \text{NIL}$ and $p_2 \neq \text{NIL}$ ex

if $[p_1].\text{info} \leq [p_2].\text{info}$ then

selected $\leftarrow p_1$

$p_1 \leftarrow [p_1].\text{next}$

else

selected $\leftarrow p_2$

$p_2 \leftarrow [p_2].\text{next}$

if result.head = NIL then

result.head $\leftarrow$ selected

last $\leftarrow$ selected

else

$[last].\text{next} \leftarrow$ selected

last $\leftarrow$ selected

if $p_1 \neq \text{NIL}$ then

rest $\leftarrow p_1$

else

rest $\leftarrow p_2$

↳

if result.head = NIL then

result.head $\leftarrow$ rest

else

$[last].\text{next} \leftarrow$ rest

↳

runs at
first iteration
(when list is empty)

for any other
node after
the head

append to result
the selected node!

sll1.head $\leftarrow \text{NIL}$

sll2.head $\leftarrow \text{NIL}$